

# B1GMB42 Software Manual

IndianaDell workspace

2026-07-05

## B1GMB42 Software Manual

**Machine:** Dell Precision Tower 5810 (B1GMB42)

**Hostname:** Tower5810

**OS:** Ubuntu 26.04 LTS (resolute)

**Workspace:** ~/Documents/IndianaDell

**Companion hardware manual:** B1GMB42-slot-port-inventory.md (slots, GPUs, storage, PERC, ports)

This manual documents every **host-facing install** the IndianaDell workspace provides: apt packages, rustup, Python venvs, built tools, Flatpak apps, GNOME preferences, Plymouth themes, optional GPU/ROCm tooling, ZFS recovery, Ventoy live persistence, and GitHub sync. Each chapter covers one topic using the same structure:

1. What gets installed
2. How it is installed
3. How to verify
4. How to customize
5. What bin/rebuild-machine does and does not do

**Build PDFs:** bin/build-all-docs (all manuals) or bin/build-software-manual (this book only).

**Quick reference:** docs/features-available.md (cheat sheet, not a replacement for this manual).

**GitHub:** <https://github.com/webaugur/IndianaDell> (private)

**Supersedes:** flat B1GMB42-software-inventory.md (now a stub with links here).

# Chapter 1 — Introduction

## What this workspace installs

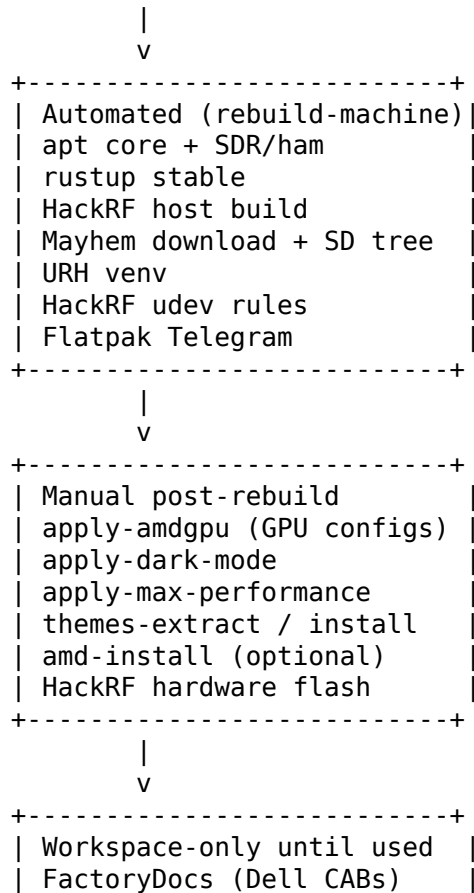
IndianaDell is a **software restoration toolkit** for Tower5810 after a fresh Ubuntu 26.04 install. It does not partition disks, configure ZFS, flash BIOS, or install Windows. It restores the development, SDR, ham radio, HackRF/Mayhem, documentation, and workstation utility stack documented in the chapters that follow.

The workspace lives at `~/Documents/IndianaDell`. Copy or clone it before running `bin/rebuild-machine`.

## Install layering

Software arrives in three layers. Understanding the order prevents skipped steps after a reinstall.

Fresh Ubuntu 26.04





**Automated** steps run via `bin/rebuild-machine` (see Chapter 2). **Manual** steps are intentional: GPU session files, GNOME gsettings, Plymouth overlay, and hardware flashing need user context or `sudo` at the right time. **Workspace-only** content stays in the repo until you invoke the matching `bin/launcher`.

## Workspace vs host paths

| Location                              | Role                                                              |
|---------------------------------------|-------------------------------------------------------------------|
| <code>~/Documents/IndianaDell/</code> | Source of truth for scripts, themes, HackRF assets                |
| <code>/usr/</code>                    | Apt-installed binaries, Plymouth themes, udev rules (after apply) |
| <code>~/ .cargo/</code>               | Rust toolchain (rustup)                                           |
| <code>hackrf/venv-urh/</code>         | Universal Radio Hacker Python venv                                |
| <code>hackrf/build/</code>            | HackRF host tools built from source                               |
| <code>hackrf/local/</code>            | Optional CMAKE_INSTALL_PREFIX for built libhackrf                 |
| <code>Themes/*/mirror/</code>         | Frozen copies of apt-owned theme files                            |
| <code>FactoryDocs/</code>             | Dell vendor packages (not auto-installed to host)                 |

## Reading guide

| If you need...               | Read          |
|------------------------------|---------------|
| Full restore after reinstall | Ch. 2 + Ch. 3 |
| Python, Rust, pandoc         | Ch. 4         |
| Boot/login/desktop look      | Ch. 5 + Ch. 7 |
| FirePro GPUs, ROCm           | Ch. 6         |
| GNU Radio, gqrx, SoapySDR    | Ch. 8         |

| If you need...                          | Read                            |
|-----------------------------------------|---------------------------------|
| fldigi, WSJT-X, CHIRP                   | Ch. 9                           |
| HackRF, Mayhem, URH                     | Ch. 10                          |
| Telegram                                | Ch. 11                          |
| iotest, dellmerge                       | Ch. 12                          |
| Dell driver CABs                        | Ch. 13                          |
| Known gaps                              | Ch. 14                          |
| Ventoy live persistence, Grok autostart | Ch. 15                          |
| ZFS rpool recovery                      | mount-rpool-recovery.sh, Ch. 15 |
| All bin/ commands                       | Appendix A                      |
| All apt package names                   | Appendix B                      |

## PATH and launchers

IndianaDell bin/ and scripts/ directories are prepended to PATH via ~/.config/indianadell/path.sh (sourced from ~/.bashrc). Project tools override same-named system binaries.

## Related documents

- **Hardware:** BIGMB42-slot-port-inventory.md + PDF — GPUs, PERC, bays, ports
- **Themes deep-dive:** Themes/README.md and per-folder READMEs (27 files)
- **HackRF inventory:** hackrf/MANIFEST.txt
- **Apt snapshots:** apt-full-manifest.txt, apt-hamradio-dev-manifest.txt
- **Rebuild log:** scripts/rebuild/last-run.log

## Chapter 2 — Rebuild and Recovery

### What gets installed

bin/rebuild-machine restores the full automated software stack in one run (~15–30 minutes, network dependent). It installs **91 apt packages** from scripts/rebuild/package-lists.sh (APT\_CORE 38 + APT\_SDR\_HAM 53), plus rustup, HackRF repos/build, Mayhem v2.4.0 assets, URH venv, udev rules, and Flatpak Telegram.

### How it is installed

```
cd ~/Documents/IndianaDell
chmod +x bin/* scripts/rebuild/*.sh
```

```
bin/rebuild-machine # full restore
bin/rebuild-machine --verify-only # check only, no installs
```

#### Environment overrides:

| Variable            | Effect                                |
|---------------------|---------------------------------------|
| SKIP_TELEGRAM=1     | Skip Flatpak Telegram install         |
| SKIP_HACKRF_BUILD=1 | Skip cmake build of HackRF host tools |

#### Phases (from scripts/rebuild/rebuild-machine.sh):

| Phase | Action                                                                             |
|-------|------------------------------------------------------------------------------------|
| 1     | apt-get update                                                                     |
| 2     | Install APT_CORE (38 packages) —<br>build, Python, docs, GPU utils, flatpak,<br>gh |
| 3     | Install APT_SDR_HAM (53 packages) —<br>GNU Radio, ham, SDR hardware,<br>HackRF     |
| 4     | Flatpak remote +<br>org.telegram.desktop (unless<br>skipped)                       |
| 5     | rustup stable if rustc missing                                                     |
| 6     | Clone HackRF/Mayhem/URH repos<br>under hackrf/repos/                               |
| 7     | Build HackRF host tools to<br>hackrf/build/ (unless skipped)                       |
| 8     | Download Mayhem v2.4.0 + extract<br>SD card tree                                   |
| 9     | Create hackrf/venv-urh/ with URH                                                   |
| 10    | Install HackRF udev rules; chmod<br>bin/ and scripts                               |
| 11    | Regenerate apt manifests; run<br>verify_stack                                      |

**Debconf preseed:** xastir/install-setuid is set to false before apt to avoid interactive hangs.

**Log file:** scripts/rebuild/last-run.log

#### How to verify

```
bin/rebuild-machine --verify-only
verify_stack checks:
```

- Every package in APT\_CORE and APT\_SDR\_HAM via dpkg-query
- Commands: rustc, cargo, gnuradio-config-info, gcc, gqrx, fldigi, wsjtx, chirpw, hackrf\_info, inspectrum, pandoc, xelatex, vkcube
- hackrf/venv-urh/bin/urh
- Mayhem firmware zip and extracted SD tree
- Built hackrf/build/hackrf-tools/src/hackrf\_sweep
- Launchers: dellmerge, gpu-stress, iotest, apply-amdgpu, rebuild-machine
- Flatpak Telegram (unless SKIP\_TELEGRAM=1)

Exit code 0 means all checks passed.

## How to customize

- **Add apt packages:** Edit scripts/rebuild/package-lists.sh, update Appendix B, re-run rebuild.
- **Pin HackRF/Mayhem version:** Edit hackrf/scripts/download-mayhem.sh and MANIFEST; rebuild does not auto-upgrade pinned releases.
- **Skip heavy steps:** Use SKIP\_\* env vars for CI or partial recovery.

## What rebuild does / does not do

| Rebuild <b>does</b>             | Rebuild <b>does not</b>             |
|---------------------------------|-------------------------------------|
| apt install all listed packages | Partition disks or ZFS              |
| rustup, HackRF build, Mayhem    | sudo bin/apply-amdgpu               |
| download                        |                                     |
| URH venv, udev rules            | bin/apply-dark-mode /               |
|                                 | apply-max-performance               |
| Regenerate                      | Plymouth theme install              |
| apt-full-manifest.txt           |                                     |
| chmod workspace scripts         | Flash HackRF / PortaPack firmware   |
|                                 | bin/amd-install (ROCm)              |
|                                 | Install FactoryDocs CABs to Windows |

After a successful rebuild, continue with **Chapter 3 — Post-Rebuild Checklist**.

## ZFS rpool / bpool recovery

**Full guide:** docs/B1GMB42-zfs-recovery.md + B1GMB42-zfs-recovery.pdf (also on **DOSBOOT** IndianaDell/recovery/).

**Quick (with scripts, from Ventoy live):**

```

sudo apt-get install -y zfsutils-linux
cd ~/Documents/IndianaDell      # or DOSBOOT/IndianaDell/recovery
sudo ./mount-rpool-recovery.sh mount
sudo ./scripts/recovery/mount-bpool-recovery.sh mount
sudo ./mount-rpool-recovery.sh chroot

```

**Without scripts:** manual `zpool import -R /recovery rpool` — see ZFS recovery manual Section 3.

Build PDF: `bin/build-zfs-recovery-doc`. Deploy to DOSBOOT: `bin/deploy-dosboot-recovery`.

See also **Chapter 15** for Ventoy live boot.

## Chapter 3 — Post-Rebuild Checklist

`bin/rebuild-machine` intentionally stops before steps that need a logged-in desktop, a reboot, or hardware attached. Run this checklist once per fresh install.

### 1. GPU session configuration

Tower5810 has three AMD FirePro cards (W5000/W5100). Multi-GPU Wayland/X11 configs live in `etc/`.

```

cd ~/Documents/IndianaDell
sudo bin/apply-amdgpu
sudo reboot

```

**Verify after reboot:** `echo $WAYLAND_DISPLAY, glxinfo -B, vkcube` on each display if needed.

See Chapter 6 for ROCm (`bin/amd-install`) — optional and not supported for ML on these GPUs.

### 2. GNOME session preferences

Run as the **desktop user** (not root):

```

bin/apply-dark-mode           # Yaru-dark GTK, shell, icons, GDM greeter
bin/apply-max-performance    # no suspend, dimming, or night light

```

**Verify:**

```

gsettings get org.gnome.desktop.interface color-scheme
powerprofilesctl get

```

See Chapter 7 for every `gsettings` key touched.

### 3. Boot splash (optional)

Default Ubuntu **bgrt** Plymouth theme shows Dell BGRT center + Ubuntu watermark. To customize:

```
bin/themes-extract                # refresh mirrors + extract logos
# edit Themes/boot/overlay/watermark.png or background.png
sudo bin/themes-install-boot      # or --oem / --no-watermark
sudo reboot
```

Restore factory: `sudo bin/themes-restore-boot`

See Chapter 5.

### 4. HackRF hardware (when device is available)

```
source bin/hackrf-env
hackrf_info                       # should list board
bin/hackrf-flash-mayhem           # extract USB flash bundle
# follow bundle README for DFU flash
bin/hackrf-prepare-sdcard        # ensure SD tree is extracted
# copy hackrf/sd-card/mayhem-v2.4.0/* to FAT32 microSD root
```

**Verify:** `hackrf_info`, on-device Mayhem version, SD apps visible on PortaPack.

See Chapter 10.

### 5. Documentation PDFs

```
bin/build-all-docs               # software manual + hardware + inventory PDFs
# or:
bin/build-software-manual         # this manual only
```

Outputs: `B1GMB42-software-manual.pdf`, `B1GMB42-slot-port-inventory.pdf`, `B1GMB42-software-inventory.pdf`.

### 6. Machine inventory baseline

```
bin/dellmerge > blgmb42.report
sudo bin/iotest                   # optional storage survey
bin/gpu-stress 60 vkcube          # optional GPU smoke test
```

### 7. FactoryDocs recovery (optional)

Only 19 of 101 pre-crash Dell packages are on disk. Re-download per `FactoryDocs/README.md` and `MANIFEST-pre-crash.txt`. These are **workspace archives**, not installed by rebuild.

See Chapter 13.

## Quick verification block

```
cd ~/Documents/IndianaDell
bin/rebuild-machine --verify-only
source bin/hackrf-env
. ~/.cargo/env && rustc --version
gnuradio-config-info --version
bin/urh --version
```

## Summary table

| Step                    | Command                          | Reboot? |
|-------------------------|----------------------------------|---------|
| GPU configs             | sudo bin/apply-amdgpu            | Yes     |
| Dark mode               | bin/apply-dark-mode              | No      |
| Max performance         | bin/apply-max-performance        | No      |
| Custom boot             | sudo bin/themes-install-boot     | Yes     |
| HackRF flash            | bin/hackrf-flash-mayhem + DFU    | Maybe   |
| ROCm (optional)         | bin/amd-install                  | Yes     |
| All doc PDFs            | bin/build-all-docs               | No      |
| Ventoy persistence seed | ~/bin/seed-ventoy-persistence.sh | No      |

## Chapter 4 — Development Toolchain

### What gets installed

| Component                            | Version / path       | Install source                                                  |
|--------------------------------------|----------------------|-----------------------------------------------------------------|
| Python<br>pip, venv                  | 3.14 (system)<br>apt | Ubuntu base + apt<br>python3-pip,<br>python3-venv               |
| numpy, scipy,<br>matplotlib<br>PyQt5 | apt<br>apt           | science stack<br>GNU Radio Companion,<br>some tools             |
| Rust<br>C/C++ build                  | 1.96.1 stable<br>apt | rustup to ~/.cargo/bin<br>build-essential,<br>cmake, pkg-config |
| Clang/LLVM                           | apt                  | bindgen-style Rust,<br>native tooling                           |
| SSL/USB/FFTW/Volk apt dev libs       |                      | SDR and native builds                                           |

| Component         | Version / path | Install source                                     |
|-------------------|----------------|----------------------------------------------------|
| ARM cross-compile | apt            | PortaPack Mayhem firmware<br>(gcc-arm-none-eabi)   |
| pandoc + XeLaTeX  | apt            | Manual PDF generation                              |
| Git, curl, wget   | apt            | repos and downloads                                |
| GitHub CLI (gh)   | 2.46 (apt)     | HTTPS/token fallback; gh auth login for API access |

**Python bindings verified on this host:** gnuradio, SoapySDR, Hamlib (capital H in Python).

**Project venv:** URH lives in hackrf/venv-urh/ (Chapter 10), not system-wide.

## How it is installed

**Apt (rebuild Phase 2):** packages listed in Appendix B under “Development” and shared dev libraries.

**Rust (rebuild Phase 5):**

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y --default-source ~/.cargo/env
```

Rebuild skips rustup if rustc is already on PATH.

## How to verify

```
python3 --version
python3 -c "import numpy, scipy, matplotlib; print('OK')"
. ~/.cargo/env && rustc --version && cargo --version
cmake --version | head -1
arm-none-eabi-gcc --version | head -1
pandoc --version | head -1
xelatex --version | head -1
```

## How to customize

- **New Python project venv:** `python3 -m venv myproject/.venv`  
`&& . myproject/.venv/bin/activate`
- **Rust toolchain:** `rustup toolchain install stable, rustup component add clippy`

- **Per-project Rust SDR crates:** `cargo add rtlsdr` etc. — no workspace scaffold ships with IndianaDell
- **Manual PDFs:** `bin/build-all-docs` (all docs) or `bin/build-software-manual` (software manual only)

## What rebuild does / does not do

| Does                                 | Does not                                     |
|--------------------------------------|----------------------------------------------|
| Install all dev apt packages         | Create application-specific venvs beyond URH |
| Install rustup if missing            | Pin a non-stable Rust toolchain              |
| <code>chmod</code> workspace scripts | Install IDE or editor plugins                |

## Chapter 5 — Themes (Boot, Login, Desktop)

### What gets installed

The **Themes/** module (~194 MB with mirrors) documents and customizes three visual stages:

| Stage          | What you see                                  | Apt packages                                                                             | Workspace folder             |
|----------------|-----------------------------------------------|------------------------------------------------------------------------------------------|------------------------------|
| <b>Boot</b>    | Dell BGRT center + spinner + Ubuntu watermark | <code>plymouth</code> ,<br><code>plymouth-theme-spinner</code> ,<br>...                  | <code>Themes/boot/</code>    |
| <b>Login</b>   | GDM on GNOME Shell                            | <code>gdm3</code> ,<br><code>gnome-shell</code> ,<br><code>yaru-theme-gnome-shell</code> | <code>Themes/login/</code>   |
| <b>Desktop</b> | Yaru GTK, icons, shell                        | <code>yaru-theme-gtk</code> ,<br><code>yaru-theme-icon</code> , ...                      | <code>Themes/desktop/</code> |

**Active Plymouth theme:** `bgrrt at /usr/share/plymouth/themes/bgrrt/bgrrt.plymouth`

### Extracted boot logos:

- `Themes/boot/extracted/bgrrt-firmware-oem.png` — Dell from UEFI BGRT (`/sys/firmware/acpi/bgrrt/image`)
- `Themes/boot/extracted/ubuntu-watermark-dark.png` — bottom Ubuntu text

**Custom Plymouth install target:** indianadell theme under /usr/share/plymouth/themes/indianadell

Each subfolder has its own README.md and apt-packages.txt. See Themes/MANIFEST.txt for a one-page map.

## How it is installed

Themes are **not** applied by bin/rebuild-machine. Use launchers:

```
bin/themes-extract           # snapshot apt-owned files + extract logos (~193 MB)
sudo bin/themes-install-boot # install custom Plymouth from boot/overlay/
sudo bin/themes-restore-boot # revert to stock bgrt
bin/apply-dark-mode          # login + desktop dark (Chapter 7)
```

### Boot overlay workflow:

```
cp my-logo.png Themes/boot/overlay/watermark.png # bottom Ubuntu text only
sudo bin/themes-install-boot
```

```
cp my-splash.png Themes/boot/overlay/background.png
sudo bin/themes-install-boot --oem # replace center (disable BGR)
```

```
sudo bin/themes-install-boot --no-watermark # hide bottom logo
```

Every Plymouth install runs update-initramfs -u — **reboot required** to preview.

## How to verify

```
plymouth-set-default-theme -l
plymouth-set-default-theme # shows active theme
ls Themes/boot/extracted/
gsettings get org.gnome.desktop.interface gtk-theme # after apply-dark-mode
```

## How to customize

Deep documentation lives in module READMEs — this chapter summarizes:

- **Boot:** Themes/boot/README.md — overlay/, stock/, indianadell/ plymouth definition
- **Login:** Themes/login/README.md — GDM greeter assets mirrored from apt
- **Desktop:** Themes/desktop/README.md — GTK/icon/shell Yaru variants

**Center Dell logo** comes from BIOS BGRT firmware, not an Ubuntu file. Change it in BIOS setup or use --oem with your PNG.

## What rebuild does / does not do

| Does                                             | Does not                                                                                                            |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Install plymouth, gdm3, Yaru via apt (Phase 2-3) | Run themes-extract or themes-install-boot<br>Set dark mode or GDM greeter prefs<br>Copy overlay PNGs into initramfs |

Run Chapter 3 checklist items 2 and 3 after rebuild.

## Chapter 6 — GPU and Display

### What gets installed

| Component                                                     | Source                      | Purpose                                                                  |
|---------------------------------------------------------------|-----------------------------|--------------------------------------------------------------------------|
| vulkan-tools,<br>mesa-utils, clinfo<br>etc/ multi-GPU configs | apt (APT_CORE)<br>workspace | Vulkan/OpenGL/OpenCL<br>diagnostics<br>Wayland, X11, udev,<br>GDM tweaks |
| amd-radeon/ scripts                                           | workspace                   | Optional ROCm driver<br>install                                          |
| bin/gpu-stress                                                | workspace                   | 3-GPU Vulkan smoke<br>test                                               |

**Hardware (this machine):** 2x AMD FirePro W5000 + 1x FirePro W5100. Vulkan and OpenCL work for graphics/compute smoke tests. **ROCm ML/HIP is not supported** on these cards (see Chapter 14).

### How it is installed

**Apt (automated):** GPU utility packages install during rebuild Phase 2.

**Session configs (manual):**

```
sudo bin/apply-amdgpu    # runs etc/apply.sh
sudo reboot
```

etc/apply.sh installs:

- etc/environment.d/99-amdgpu-wayland.conf
- etc/X11/xorg.conf.d/20-amdgpu-multi-gpu.conf
- etc/modprobe.d/amdgpu-multigpu.conf
- etc/udev/rules.d/99-amdgpu-multigpu.rules

- `etc/profile.d/amdgpu-multigpu.sh`
- `etc/gdm3/custom.conf` (if present)

### Optional ROCm:

```
bin/amd-preflight      # check prerequisites
bin/amd-install        # full driver stack from amd-radeon/
bin/amd-verify
bin/amd-uninstall      # remove if needed
```

### How to verify

```
vkcube                # Vulkan cube (per display)
clinfo | head -30      # OpenCL platforms/devices
glxinfo -B             # OpenGL renderer
bin/gpu-stress 60 vkcube # stress all GPUs ~60s
lspci -nn | grep -i vga
```

### How to customize

- Edit files under `etc/` before re-running `sudo bin/apply-amdgpu`
- ROCm install scripts and README in `amd-radeon/` — machine-specific; read preflight output
- Hardware details: `B1GMB42-slot-port-inventory.md` Video section

### What rebuild does / does not do

| Does                                             | Does not                                      |
|--------------------------------------------------|-----------------------------------------------|
| Install mesa-utils, vulkan-tools, clinfo         | Run <code>apply-amdgpu</code>                 |
| Ensure <code>bin/gpu-stress</code> is executable | Install ROCm                                  |
|                                                  | Configure monitor layout (use GNOME Settings) |

**Required post-rebuild:** `sudo bin/apply-amdgpu + reboot` (Chapter 3).

## Chapter 7 — GNOME Session

### What gets installed

GNOME desktop preferences applied via `gsettings` (user session) and optionally GDM (login greeter). No apt packages beyond what Ubuntu desktop already provides (`gdm3`, `gnome-shell`, Yaru themes).

#### Scripts:

- `scripts/gnome/apply-dark-mode.sh` via `bin/apply-dark-mode`
- `scripts/gnome/apply-max-performance.sh` via `bin/apply-max-performance`

## How it is installed

Run as the **logged-in desktop user**, not root:

`bin/apply-dark-mode`

`bin/apply-max-performance`

### apply-dark-mode

Sets:

| Schema                                               | Key                       | Value                                         |
|------------------------------------------------------|---------------------------|-----------------------------------------------|
| <code>org.gnome.desktop.interface</code>             | <code>color-scheme</code> | <code>prefer-dark</code>                      |
| <code>org.gnome.desktop.interface</code>             | <code>gtk-theme</code>    | <code>Yaru-dark (override: GTK_THEME=)</code> |
| <code>org.gnome.desktop.interface</code>             | <code>icon-theme</code>   | <code>Yaru-dark</code>                        |
| <code>org.gnome.shell.ubuntu</code>                  | <code>color-scheme</code> | <code>prefer-dark</code>                      |
| <code>org.gnome.desktop.wm.preferences</code>        | <code>theme</code>        | <code>Yaru-dark</code>                        |
| <code>org.gnome.settings-daemon.plugins.color</code> | <code>night-light</code>  | <code>disabled</code>                         |

With `APPLY_GDM=1` (default), also sets GDM greeter dark via `sudo dbus-run-session gsettings ....`

### apply-max-performance

Sets:

- Power plugin: no suspend on AC/battery, no idle dim, lid close does nothing
- Session idle delay: 0
- Screensaver: no blanking or lock on idle
- Night light: off
- `powerprofilesctl set performance` when available

## How to verify

`gsettings get org.gnome.desktop.interface color-scheme`

`gsettings get org.gnome.desktop.interface gtk-theme`

`gsettings get org.gnome.settings-daemon.plugins.power sleep-inactive-ac-type`

`powerprofilesctl get`

Log out and back in to confirm GDM greeter if dark mode was applied.

## How to customize

- Override themes: `GTK_THEME=Yaru-viridian bin/apply-dark-mode`
- Skip GDM: `APPLY_GDM=0 bin/apply-dark-mode`
- Revert individual keys with `gsettings reset ...` or GNOME Settings app
- Login/desktop asset mirrors: `Themes/login/, Themes/desktop/`

## What rebuild does / does not do

| Does                                    | Does not                                              |
|-----------------------------------------|-------------------------------------------------------|
| Install gdm3, gnome-shell, Yaru via apt | Change any gsettings<br>Set performance power profile |

Run both launchers after every fresh install (Chapter 3).

## Chapter 8 — GNU Radio and Desktop SDR

### What gets installed

| Component        | Version      | Packages / path                                                                   |
|------------------|--------------|-----------------------------------------------------------------------------------|
| GNU Radio        | 3.10.12.0    | gnuradio, gnuradio-dev, gnuradio-doc                                              |
| Companion blocks | apt          | gr-osmosdr, gr-limesdr, gr-fosphor, gr-air-modes, gr-hpsdr, gr-dab, gr-satellites |
| SoapySDR         | apt + Python | libsoapyhdr-dev, python3-soapyhdr, modules                                        |
| Hardware libs    | apt          | RTL-SDR, HackRF, Airspy, bladeRF, Lime, UHD                                       |
| Desktop apps     | apt          | gqrx-sdr, quisk, inspectrum, hacktv                                               |

**SoapySDR modules on this host:** HackRF, RTL-SDR (osmosdr), Airspy, bladeRF, Lime, MiriSDR, HydraSDR, PlutoSDR, Red Pitaya, remote, audio, UHD.

## How it is installed

All packages in APT\_SDR\_HAM (rebuild Phase 3). Dev libraries in APT\_CORE support building OOT modules.

### Typical workflow:

```
grcc myflowgraph.grc      # compile Companion graph
gqrx                      # general receiver GUI
quisk                     # transceiver GUI
inspectrum capture.cf32   # visualize IQ files
```

For HackRF-specific host tools and URH, see Chapter 10.

## How to verify

```
gnuradio-config-info --version
grcc --help | head -1
python3 -c "import gnuradio; print(gnuradio.__version__)"
python3 -c "import SoapySDR; print('SoapySDR OK')"
SoapySDRUtil --info
gqrx --version 2>/dev/null || command -v gqrx
```

With hardware attached:

```
rtl_test -t              # RTL-SDR
hackrf_info              # HackRF
```

## How to customize

- Add OOT modules: `sudo apt install gr-<name>` or build from source against `gnuradio-dev`
- GPU waterfall: `gr-fosphor` blocks in flowgraphs
- Filtered apt list: `apt-hamradio-dev-manifest.txt` (178 SDR/ham-related packages on full system)

## What rebuild does / does not do

| Does                                                    | Does not                        |
|---------------------------------------------------------|---------------------------------|
| Install full GNU Radio + gr-* stack                     | Calibrate specific SDR hardware |
| Install gqrx, quisk, inspectrum                         | Install SDRangel or SigDigger   |
| Verify gnuradio-config-info, grcc, gqrx in verify_stack | Flash firmware on SDR devices   |

## Chapter 9 — Ham Radio (Desktop)

### What gets installed

| Application | Command                     | Apt package                                                 | Role                           |
|-------------|-----------------------------|-------------------------------------------------------------|--------------------------------|
| fldigi      | <code>fldigi</code>         | <code>fldigi</code>                                         | Digital modes (PSK, RTTY, ...) |
| WSJT-X      | <code>wsjtx</code>          | <code>wsjtx, wsjtx-data</code>                              | FT8, JT65, weak-signal         |
| CHIRP       | <code>chirpw, chirpc</code> | <code>chirp</code>                                          | Radio programming              |
| direwolf    | <code>direwolf</code>       | <code>direwolf</code>                                       | Sound-card TNC / APRS          |
| gpredict    | <code>gpredict</code>       | <code>gpredict</code>                                       | Satellite pass prediction      |
| grig        | <code>grig</code>           | <code>grig</code>                                           | Hamlib rig control GUI         |
| xastir      | <code>xastir</code>         | <code>xastir, xastir-data</code>                            | APRS map client                |
| Hamlib      | API                         | <code>libhamlib-dev, libhamlib-utils, python3-hamlib</code> | Rig control library            |

### How it is installed

Apt packages in `APT_SDR_HAM` (rebuild Phase 3).

**xastir debconf:** rebuild preseeds `xastir/install-setuid` boolean false to avoid interactive install hangs.

```
fldigi &  
wsjtx &  
chirpw &  
direwolf -p  
gpredict &
```

```
grig &  
xastir &
```

## How to verify

```
command -v fldigi wsjtx chirpw direwolf gpredict grig xastir  
python3 -c "import Hamlib; print('Hamlib OK')"  
bin/rebuild-machine --verify-only # checks fldigi, wsjtx, chirpw
```

Configure rig control in each app via Hamlib model selection.

## How to customize

- Radio definitions: CHIRP stock configs + your radio CSV
- WSJT-X: ~/.config/WSJT-X/
- xastir maps: xastir-data package + user map sources
- direwolf: ~/.direwolf/direwolf.conf

## What rebuild does / does not do

| Does                                  | Does not                          |
|---------------------------------------|-----------------------------------|
| Install all ham desktop apps + Hamlib | Configure radios or call signs    |
| Preseed xastir setuid prompt          | Set up APRS IS or igates          |
| Verify fldigi, wsjtx, chirpw commands | Install fldigi/WSJT-X from source |

# Chapter 10 — HackRF and PortaPack Mayhem

## What gets installed

**Recommended firmware:** PortaPack Mayhem v2.4.0

### Apt packages

hackrf, hackrf-firmware, libhackrf-dev, hackrf-doc, inspectrum, hacktv, dfu-util, openocd, ARM GCC toolchain, plus GNU Radio/SoapySDR deps (Chapter 8).

### Built from source (hackrf/build/)

| Tool                                                 | Notes                                                                           |
|------------------------------------------------------|---------------------------------------------------------------------------------|
| hackrf_sweep                                         | Spectrum sweep — not in older apt splits                                        |
| hackrf_info,<br>hackrf_transfer, ...<br>libhackrf.so | Newer libhackrf (0.10.0) than apt alone<br>Under<br>hackrf/build/libhackrf/src/ |

Install prefix: `hackrf/local/` (`CMAKE_INSTALL_PREFIX`).

### Release assets (`hackrf/releases/`)

| File                                | Size   | Purpose           |
|-------------------------------------|--------|-------------------|
| FIRMWARE_mayhem_v2.4.0.zip          | 2.4 MB | USB flash bundle  |
| COPY_TO_SDCARD_hackrf_mayhem_v2.4.0 | 201 MB | PortaPack microSD |
| no-world-map.zip                    |        |                   |
| OCI_hackrf_mayhem_v2.4.0.ppfw.tar   | 2.2 MB | Web flasher image |

**Extracted SD tree:** `hackrf/sd-card/mayhem-v2.4.0/` (276 MB, 84 apps in APPS/)

### Source repos (`hackrf/repos/`)

hackrf, mayhem-firmware (+ submodules), portapack-hackrf, urh, hacktv

### Python venv

`hackrf/venv-urh/` — Universal Radio Hacker **2.10.0** (PyQt6). Launch: `bin/urh`

### udev

`hackrf/scripts/99-hackrf.rules` installed to `/etc/udev/rules.d/` — plugdev access.

## How it is installed

### Automated (rebuild Phases 6–10):

1. Clone repos (shallow, skip if `.git` exists)

2. Init Mayhem submodules if needed
3. cmake + build HackRF host (skip: SKIP\_HACKRF\_BUILD=1)
4. hackrf/scripts/download-mayhem.sh
5. hackrf/scripts/prepare-sdcard.sh
6. URH venv if missing
7. hackrf/scripts/setup-udev.sh

### Manual (hardware present):

```
source bin/hackrf-env
bin/hackrf-flash-mayhem           # extract flash bundle
bin/hackrf-prepare-sdcard        # re-extract SD payload
bin/hackrf-build-mayhem          # compile Mayhem from source (advanced)
bin/hackrf-download-mayhem       # re-fetch releases
```

**PATH:** source bin/hackrf-env adds hackrf/build/hackrf-tools/src and hackrf/local/bin.

### How to verify

```
bin/rebuild-machine --verify-only
source bin/hackrf-env
hackrf_info           # needs USB device
hackrf/build/hackrf-tools/src/hackrf_sweep --help | head -1
bin/urh --version
ls hackrf/sd-card/mayhem-v2.4.0/APPS | wc -l
```

### How to customize

- **Upgrade Mayhem:** Edit hackrf/scripts/download-mayhem.sh version URLs; re-run download + prepare
- **SD apps:** Copy subsets from sd-card/mayhem-v2.4.0/APPS/ to microSD
- **Full inventory:** hackrf/MANIFEST.txt

### Mayhem v2.4.0 highlights

On-device: Morse RX/TX, RTTY, FPV detect, ADSB, ACARS, BLE, TPMS, KeeLoq, EPIRB, SAME, MDC-1200, P25, KISS TNC, Looking Glass, SubGHz, Flipper TX, waterfall designer, time sink.

SD apps include: fpv\_detect, kiss\_tnc, keeloqtx, siggen, fmradio, sstvrx, wardrivemap, waterfall\_designer, and more.

### What rebuild does / does not do

| Does                                      | Does not                            |
|-------------------------------------------|-------------------------------------|
| Clone repos, build tools, download Mayhem | DFU-flash firmware to hardware      |
| Create URH venv, install udev             | Format or write microSD in a reader |
| Verify zip, SD tree, hackrf_sweep         | Test with HackRF USB attached       |

## Chapter 11 — Flatpak Applications

### What gets installed

| App      | Flatpak ID           | Version (this host) |
|----------|----------------------|---------------------|
| Telegram | org.telegram.desktop | 6.9.3               |

**Runtime dependency:** flatpak package from APT\_CORE.

### How it is installed

Rebuild Phase 4:

```
flatpak remote-add --if-not-exists flathub https://dl.flathub.org/repo/flathub.flatpakrepo
flatpak install -y flathub org.telegram.desktop
```

Skip with SKIP\_TELEGRAM=1 bin/rebuild-machine.

### How to verify

```
flatpak list --app | grep telegram
flatpak run org.telegram.desktop --version 2>/dev/null || true
bin/rebuild-machine --verify-only
```

### How to customize

```
flatpak update org.telegram.desktop
flatpak override --user org.telegram.desktop ... # permissions, env
bin/apply-dark-mode sets prefer-dark color scheme; Flatpak GTK4
apps pick this up via portal when supported.
```

## What rebuild does / does not do

| Does                                                       | Does not                                     |
|------------------------------------------------------------|----------------------------------------------|
| Install flatpak via apt                                    | Install SDRangel, SigDigger (not on Flathub) |
| Add flathub remote + Telegram                              | Pin Telegram to a specific commit            |
| Treat Telegram miss as non-fatal on install (warns in log) | Install other Flatpak apps by default        |

## Chapter 12 — Machine Utilities

### What gets installed

Workspace scripts for inventory, storage benchmark, and GPU stress — no dedicated apt packages beyond shared GPU utils (mesa-utils, vulkan-tools).

| Utility        | Launcher       | Script                    | Output                         |
|----------------|----------------|---------------------------|--------------------------------|
| Dell inventory | bin/dellmerge  | scripts/dell/dellmerge.sh | dellmerge.sh<br>*.report files |
| Storage survey | bin/iotest     | scripts/storage/iotest.sh | iotest.sh<br>(sudo)            |
| GPU stress     | bin/gpu-stress | scripts/gpu/gpu-stress.sh | Vulkan/EGL<br>per GPU          |

**Example reports in workspace:** blgmb42.report, B1GMB42.ioperf (from prior runs).

### How it is installed

Scripts ship with the workspace. Rebuild Phase 10 runs `chmod +x` on `bin/*` and `scripts/*/*.sh`.

```
bin/dellmerge > blgmb42.report
sudo bin/iotest
bin/gpu-stress 60 vkcube
```

### How to verify

```
[[ -x bin/dellmerge && -x bin/iotest && -x bin/gpu-stress ]] && echo OK
bin/rebuild-machine --verify-only # checks dellmerge, gpu-stress, iotest, apply
head -20 blgmb42.report 2>/dev/null || bin/dellmerge | head -20
```

## How to customize

- Edit scripts/dell/dellmerge.sh to add inventory fields
- gpu-stress accepts duration and backend (vkcube default)
- iotest targets block devices — read script header before running on production pools

## What rebuild does / does not do

| Does                              | Does not                              |
|-----------------------------------|---------------------------------------|
| chmod utility launchers           | Run dellmerge or iotest automatically |
| Verify launcher executables exist | Archive reports to a fixed path       |

## Chapter 13 — FactoryDocs (Workspace Archive)

### What gets installed

**Nothing on the Linux host automatically.** FactoryDocs is a sorted archive of Dell T5810 vendor support packages for Windows recovery, firmware, and reference — stored only in the workspace.

| Metric                       | Pre-crash | Current        |
|------------------------------|-----------|----------------|
| Sorted packages              | 101       | <b>19</b>      |
| GPU drivers (FirePro/Quadro) | Yes       | <b>Missing</b> |
| PERC H710 driver/firmware    | Yes       | <b>Missing</b> |
| Audio / input drivers        | Yes       | <b>Missing</b> |
| Win7/10/WinPE CAB packs      | Yes       | <b>Missing</b> |

Full pre-crash file list: FactoryDocs/MANIFEST-pre-crash.txt (91 items flagged MISS).

### Layout

| Folder           | Contents                                           |
|------------------|----------------------------------------------------|
| System-T5810/    | BIOS, chipset, ME, TPM, manuals                    |
| GPU/             | AMD FirePro, NVIDIA ( <b>empty — re-download</b> ) |
| Storage/         | PERC H710, Intel RST, SSD/HDD firmware             |
| Network/         | Intel Ethernet                                     |
| Dell-Management/ | Command Update, Configure                          |

| Folder           | Contents                     |
|------------------|------------------------------|
| Expansion-Cards/ | Serial, Thunderbolt docs     |
| _Misc/           | Non-Dell packages            |
| _incoming/       | Drop new Dell downloads here |

## How it is installed

### Ingest new downloads:

```
cp ~/Downloads/* ~/Documents/IndianaDell/FactoryDocs/_incoming/
python3 ~/Documents/IndianaDell/FactoryDocs/_sort_factory_docs.py
```

### Priority re-downloads from Dell T5810 drivers:

1. GPU/AMD-FirePro/Windows/ — Video\_Driver\_C5FPW (W5000/W5100)
2. Storage/RAID-Controller-PERC/Windows/ — PERC H710
3. System-T5810/Windows-10/ — T5810-win10 CAB
4. Audio/Windows/ — Audio\_Driver\_5P33P
5. Dell-Management/Windows/ — Command Configure, Monitor

Windows installs use Dell CAB/EXE packages from these folders — not apt.

## How to verify

```
find FactoryDocs -type f ! -path '*/_incoming/*' | wc -l
cat FactoryDocs/README.md
grep MISS FactoryDocs/MANIFEST-pre-crash.txt | wc -l
```

## How to customize

- `_sort_factory_docs.py` dedupes and sorts by hardware category
- Cross-reference hardware manual: `B1GMB42-slot-port-inventory.md` for what hardware needs drivers

## What rebuild does / does not do

| Does    | Does not                                                                             |
|---------|--------------------------------------------------------------------------------------|
| Nothing | Copy FactoryDocs to system paths<br>Install Windows drivers<br>Download missing CABs |

FactoryDocs recovery is a **manual** ongoing task (Chapter 3 item 7).

## Chapter 14 — Gaps and Limits

Documented boundaries of what IndianaDell does **not** install or support on this host.

### Not installed

| Item                      | Notes                                                                    |
|---------------------------|--------------------------------------------------------------------------|
| SDRangel, SigDigger       | Not on Flathub; use gqrx + URH + inspectrum                              |
| Rust SDR crate workspace  | Toolchain only — add crates per project with <code>cargo add</code>      |
| ZFS / disk layout tools   | Out of scope — handled at OS install time                                |
| Windows / dual-boot       | FactoryDocs holds CABs; no auto-install                                  |
| HackRF hardware test      | No device attached at last verify (2026-07-05)                           |
| Ventoy seed on every boot | Manual — run <code>~/bin/seed-ventoy-persistence.sh</code> after changes |

### Lost in TPM/ZFS crash

| Item                        | Recovery                                                                       |
|-----------------------------|--------------------------------------------------------------------------------|
| Pre-crash full apt list     | <b>Not found</b> — use <code>apt-full-manifest.txt</code> from current rebuild |
| Pre-crash FactoryDocs       | 82/101 packages missing — see <code>MANIFEST-pre-crash.txt</code>              |
| Pre-crash apt-hamradio list | Regenerated by <code>rebuild save_manifests()</code>                           |

### GPU / ROCm matrix

|                                |                                                                    |
|--------------------------------|--------------------------------------------------------------------|
| Capability                     | W5000 / W5100 on Ubuntu 26.04                                      |
| Desktop amdgpu Vulkan (vkcube) | Yes — with etc/ configs<br>Yes                                     |
| OpenCL (clinfo)                | Yes (limited)                                                      |
| ROCm HIP / ML training         | <b>No</b> — not in AMD ROCm support matrix for FirePro W5000/W5100 |

Use `bin/amd-preflight` before `bin/amd-install`; expect warnings for these GPUs.

## Rebuild intentional omissions

These remain **manual** by design (see Chapter 3):

- `sudo bin/apply-amdgpu`
- `bin/apply-dark-mode`, `bin/apply-max-performance`
- `bin/themes-extract`, `sudo bin/themes-install-boot`
- `bin/hackrf-flash-mayhem` (DFU with hardware)
- `bin/amd-install` (optional ROCm)

Future enhancement could fold GNOME/theme steps into rebuild; current manual documents the gap explicitly.

## Encryption / TPM

Hardware manual recommends **not** re-enabling ZFS encryption until TPM + recovery strategy is documented. IndianaDell rebuild does not touch encryption.

## When something fails

1. Read `scripts/rebuild/last-run.log`
2. Run `bin/rebuild-machine --verify-only` for targeted MISS lines
3. Re-run individual phases (`apt`, HackRF build, Mayhem download) manually
4. Check chapter-specific verify sections

## Reporting issues

Capture:

```
bin/dellmerge > debug.report
bin/rebuild-machine --verify-only 2>&1 | tee verify.log
uname -a && lsb_release -a
```

## Chapter 15 — Ventoy Live Session & Persistence

Portable Ubuntu 26.04 on the **Wiggly** Ventoy stick, with a writable overlay so login state, apps, Grok, and IndianaDell survive reboots.

## What gets persisted

| Item                  | Location (live boot)                    | Seeded to casper image                 |
|-----------------------|-----------------------------------------|----------------------------------------|
| User home             | /home/ubuntu                            | cow/upper/home/ubuntu/                 |
| Installed packages    | dpkg overlay                            | cow/upper/var/lib/dpkg/                |
| GDM autologin         | /etc/gdm3/custom.conf                   | cow/upper/etc/gdm3/                    |
| Grok auth + sessions  | ~/.grok/                                | same (never in git)                    |
| GitHub CLI auth       | ~/.config/gh/                           | same                                   |
| SSH keys              | ~/.ssh/                                 | same                                   |
| <b>Runtime source</b> | /home/user/ when ZFS rpool is available | pulled at login via resolve-secrets.sh |
| IndianaDell workspace | ~/Documents/IndianaDell workspace       | same (git clone or rsync)              |
| PATH overrides        | ~/.config/indianadell/path overrides    | same                                   |

**Persistence image:** /persistence/ubuntu-26.04.dat (24 GB ext4, label casper-rw) on the internal Ventoy exFAT volume (**Wiggly**, sdc1).

**Ventoy config:** ventoy/ventoy.json maps ubuntu-26.04-desktop-amd64.iso to that .dat file with autosel: 1. Canonical copy in scripts/ventoy/ventoy.json.

## How it is installed

**One-time setup from Tower5810** (Wiggly mounted at /mnt/wiggly):

```
sudo mount -o uid=$(id -u),gid=$(id -g) /dev/disk/by-label/Wiggly /mnt/wiggly
bin/setup-wiggly-ventoy # verify ISO, ventoy.json, .dat filesystem
```

Extend an undersized image: sudo scripts/ventoy/ExtendPersistentImg.sh /mnt/wiggly/persistence/ubuntu-26.04.dat <MB> then resize2fs on the loop device if needed.

**Seed session state** from a running session with Wiggly mounted (e.g. /mnt/wiggly):

```
# One-time or after changes – seeds current ubuntu session into the .dat image
~/bin/seed-ventoy-persistence.sh
# or, if the image is already mounted:
PERSIST_MOUNT=/mnt/persist-check ~/bin/seed-ventoy-persistence.sh
```

The seed script copies home, dpkg/apt state, GDM autologin, SSH keys (including /home/user/.ssh/id\_rsa when present), and the IndianaDell tree.

## Login experience (configured)

1. **GDM autologin** — user ubuntu (/etc/gdm3/custom.conf)
2. **PATH** — IndianaDell bin/ and scripts/ override system (~/.config/indianadell/path.sh)
3. **Grok autostart** — Ptyxis fullscreen, resumes IndianaDell session (~/.config/autostart/grok-indianadell.desktop)

Launcher: ~/bin/grok-indianadell-launch.sh

resolve-secrets.sh materializes secrets from /home/user when rpool exists, else uses Ventoy \$HOME.

Runs ~/bin/seed-ventoy-persistence.sh **before** Grok (logs to ~/.cache/seed-ventoy.log).

Seed verifies **internet + DNS** first; if down, offers NetworkManager bring-up or skip. Default session: ~/Documents/IndianaDell (session ID in script env vars).

## ZFS recovery (rpool + bpool)

**Manual:** B1GMB42-zfs-recovery.pdf (repo root and DOSB00T/IndianaDell/recovery/).

Boot Ventoy Ubuntu live — **do not** use the broken installed system as root.

```
sudo apt-get install -y zfsutils-linux
cd ~/Documents/IndianaDell # or /media/.../DOSB00T1/IndianaDell/recovery
sudo ./mount-rpool-recovery.sh mount
sudo ./scripts/recovery/mount-bpool-recovery.sh mount
sudo ./mount-rpool-recovery.sh chroot
# repair inside chroot; then exit and umount both scripts
```

**No IndianaDell?** Same manual, Section 3 — raw zpool import commands.

Deploy kit to DOSBOOT: bin/deploy-dosboot-recovery (from Tower5810).

## GitHub repository

Full workspace (including FactoryDocs): <https://github.com/webaugur/IndianaDell> (private)

```
bin/pull-repo --verify          # IndianaDell + hackrf/repos + LFS + stack verify
bin/push-repo                  # push main (SSH default)
```

HTTPS push (optional): `INDIANADELL_REMOTE=https://github.com/webaugur/IndianaDell.git` after `gh auth login`.

Large FactoryDocs installers (>100 MB) use **Git LFS**. `bin/pull-repo` runs `git lfs pull`.

## How to verify

Boot Ventoy □ Ubuntu 26.04 (persistence auto-selected). Then:

```
findmnt / | grep -q cow && echo "persistence overlay active"
grep AutomaticLogin=ubuntu /etc/gdm3/custom.conf
echo "$INDIANADELL_ROOT"      # should be ~/Documents/IndianaDell
which dellmerge pull-repo push-repo grok
bin/pull-repo --verify
google-chrome --version
```

## How to customize

| Goal                     | Action                                                                                  |
|--------------------------|-----------------------------------------------------------------------------------------|
| Re-seed after changes    | <code>~/bin/seed-ventoy-persistence.sh</code>                                           |
| Change Grok session      | Edit <code>GROK_SESSION_ID</code> in <code>grok-indianadell-launch.sh</code>            |
| Disable autostart        | Remove <code>~/config/autostart/grok-indianadell.desktop</code> , re-seed               |
| Enlarge persistence      | <code>scripts/ventoy/ExtendPersistentImg.sh</code> (+ <code>resize2fs</code> if needed) |
| Verify/fix Ventoy layout | <code>bin/setup-wiggly-ventoy</code> from Tower5810                                     |

## What rebuild does / does not do

| Does                                                 | Does not                                  |
|------------------------------------------------------|-------------------------------------------|
| Install Chrome, gh, git-lfs when run on live session | Configure Ventoy <code>ventoy.json</code> |
| Document seed script in this chapter                 | Auto-run seed on reboot                   |

|      |                                    |
|------|------------------------------------|
| Does | Does not                           |
|      | Manage Ventoy ISO partition layout |

## Appendix A — bin/ Launchers

All launchers live in ~/Documents/IndianaDell/bin/. **PATH** is set automatically via ~/.config/indianadell/path.sh (IndianaDell tools override system binaries).

| Launcher                | Runs                                                                                                             | Chapter |
|-------------------------|------------------------------------------------------------------------------------------------------------------|---------|
| rebuild-machine         | scripts/rebuild-machine.sh                                                                                       | 2       |
| build-software-manual   | scripts/docs/build-software-manual.sh                                                                            | 1       |
| build-all-docs          | scripts/docs/build-all-docs.sh                                                                                   | 1       |
| pull-repo               | scripts/github/pull-all.sh —<br>IndianaDell +<br>nested repos +<br>LFS                                           | 15      |
| push-repo               | bin/push-repo<br>□ GitHub                                                                                        | 15      |
| setup-wiggly-ventoy     | webaur/IndianaDell<br>(SSH default)<br>scripts/ventoy/setup-wiggly-ventoy.sh —<br>ISO +<br>ventoy.json +<br>.dat | 15      |
| build-zfs-recovery-doc  | scripts/docs/build-zfs-recovery-doc.sh                                                                           | 15      |
| deploy-dosboot-recovery | scripts/recovery/deploy-to-dosboot.sh                                                                            | 15      |
| dellmerge               | scripts/dell/dellmerge.sh                                                                                        | 12      |
| gpu-stress              | scripts/gpu/gpu6, 12<br>stress.sh                                                                                | 6, 12   |
| iostat                  | scripts/storage/iostat.sh                                                                                        | 12      |

| Launcher               | Runs                                                 | Chapter |
|------------------------|------------------------------------------------------|---------|
| apply-amdgpu           | etc/apply.sh                                         | 6       |
| amd-install            | amd-radeon/install-all.sh                            | 6       |
| amd-preflight          | amd-radeon/00-preflight.sh                           | 6       |
| amd-verify             | amd-radeon/04-verify.sh                              | 6       |
| amd-uninstall          | amd-radeon/uninstall.sh                              | 6       |
| apply-dark-mode        | scripts/gnome/apply-dark-mode.sh                     | 5       |
| apply-max-performance  | scripts/gnome/apply-max-performance.sh               | 5       |
| themes-extract         | Themes/scripts/extract-all.sh                        | 5       |
| themes-install-boot    | Themes/scripts/install-boot-theme.sh                 | 5       |
| themes-restore-boot    | Themes/scripts/install-boot-theme.sh --restore-stock | 5       |
| hackrf-env             | sources                                              | 10      |
| urh                    | hackrf/scripts/env.sh                                | 10      |
| hackrf-setup-udev      | hackrf/scripts/launch-urh.sh                         | 10      |
| hackrf-download-mayhem | hackrf/scripts/setup-udev.sh                         | 10      |
| hackrf-prepare-sdcard  | hackrf/scripts/download-mayhem.sh                    | 10      |
| hackrf-flash-mayhem    | hackrf/scripts/prepare-sdcard.sh                     | 10      |
| hackrf-build-mayhem    | hackrf/scripts/flash-mayhem.sh                       | 10      |
|                        | hackrf/scripts/build-mayhem.sh                       | 10      |

**Ventoy session (scripts/ventoy/ □ ~/bin via install-ventoy-session.sh):**

| Script                     | Purpose                                               |
|----------------------------|-------------------------------------------------------|
| seed-ventoy-persistence.sh | Snapshot session into Ventoy casper image             |
| seed-network-check.sh      | Internet/DNS check before seed                        |
| grok-indianadell-launch.sh | Seed then Grok fullscreen autostart                   |
| install-ventoy-session.sh  | Install helpers, autostart, PATH                      |
| mount-rpool-recovery.sh    | ZFS rpool chroot recovery (workspace root)            |
| mount-bpool-recovery.sh    | ZFS bpool mount at /recovery/boot (scripts/recovery/) |

**Note:** hackrf-env must be **sourced**, not executed: source bin/hackrf-env

**Sudo required:** apply-amdgpu, themes-install-boot, themes-restore-boot, iotest, hackrf-setup-udev (udev install), amd-install, mount-rpool-recovery.sh

## Appendix B — Apt Packages by Chapter

**Source of truth:** scripts/rebuild/package-lists.sh (APT\_CORE + APT\_SDR\_HAM).

**Total:** 91 packages installed by bin/rebuild-machine (38 core + 53 SDR/ham).

**Full system snapshot:** apt-full-manifest.txt (~2257 packages after rebuild).

**SDR/ham filter:** apt-hamradio-dev-manifest.txt (~178 related packages).

Packages below are grouped by manual chapter. Shared dev libraries appear under Chapter 4 and are reused by Chapters 8-10.

### Chapter 4 — Development

build-essential, cmake, pkg-config, git, curl, wget, unzip, python3-pip, python3-venv, python3-dev, python3-numpy, python3-scipy, python3-matplotlib, python3-yaml, python3-requests, python3-pyqt5, python3-psutil, libssl-dev, clang, llvm-dev, libclang-dev, libusb-1.0-0-dev, libfftw3-dev, libvolk-dev, portaudio19-dev, libsndfile1-dev, libboost-dev, libboost-program-options-dev, pandoc, texlive-latex-recommended, texlive-fonts-recommended, texlive-xetex, gh

## Chapter 6 — GPU and Display

vulkan-tools, mesa-utils, mesa-utils-bin, clinfo

## Chapter 8 — GNU Radio and SDR

gnuradio, gnuradio-dev, gnuradio-doc, gr-osmosdr, gr-limesdr, gr-fosphor, gr-air-modes, gr-hpsdr, gr-dab, gr-satellites, libsoapysdr-dev, python3-soapysdr, soapysdr-module-osmosdr, soapysdr-module-mirisdr, uhd-soapysdr, rtl-sdr, librtlsdr-dev, airspy, libairspy-dev, bladerf, libbladerf-dev, limesuite, limesuite-udev, uhd-host, libuhd-dev, gqrx-sdr, quisk, inspectrum, hacktv

## Chapter 9 — Ham Radio

libhamlib-dev, libhamlib-utils, python3-hamlib, fldigi, wsjtx, wsjtx-data, chirp, direwolf, gpredict, grig, xastir, xastir-data

## Chapter 10 — HackRF and Mayhem

hackrf, hackrf-firmware, libhackrf-dev, hackrf-doc, dfu-util, openocd, gcc-arm-none-eabi, binutils-arm-none-eabi, libnewlib-arm-none-eabi, ccache, lz4, bzip2

(Also uses Chapter 8 packages for GNU Radio/SoapySDR integration.)

## Chapter 11 — Flatpak

flatpak (application org.telegram.desktop installed via flatpak, not apt)

## Chapters 5, 7, 12, 13 — No dedicated apt arrays

Themes, GNOME prefs, machine utilities, and FactoryDocs use workspace scripts or Ubuntu desktop packages already on the base install (gdm3, gnome-shell, plymouth, Yaru themes) — not enumerated separately in package-lists.sh.